Agentic Harness

Ravi Pandey

Agentic Harness: structured software development

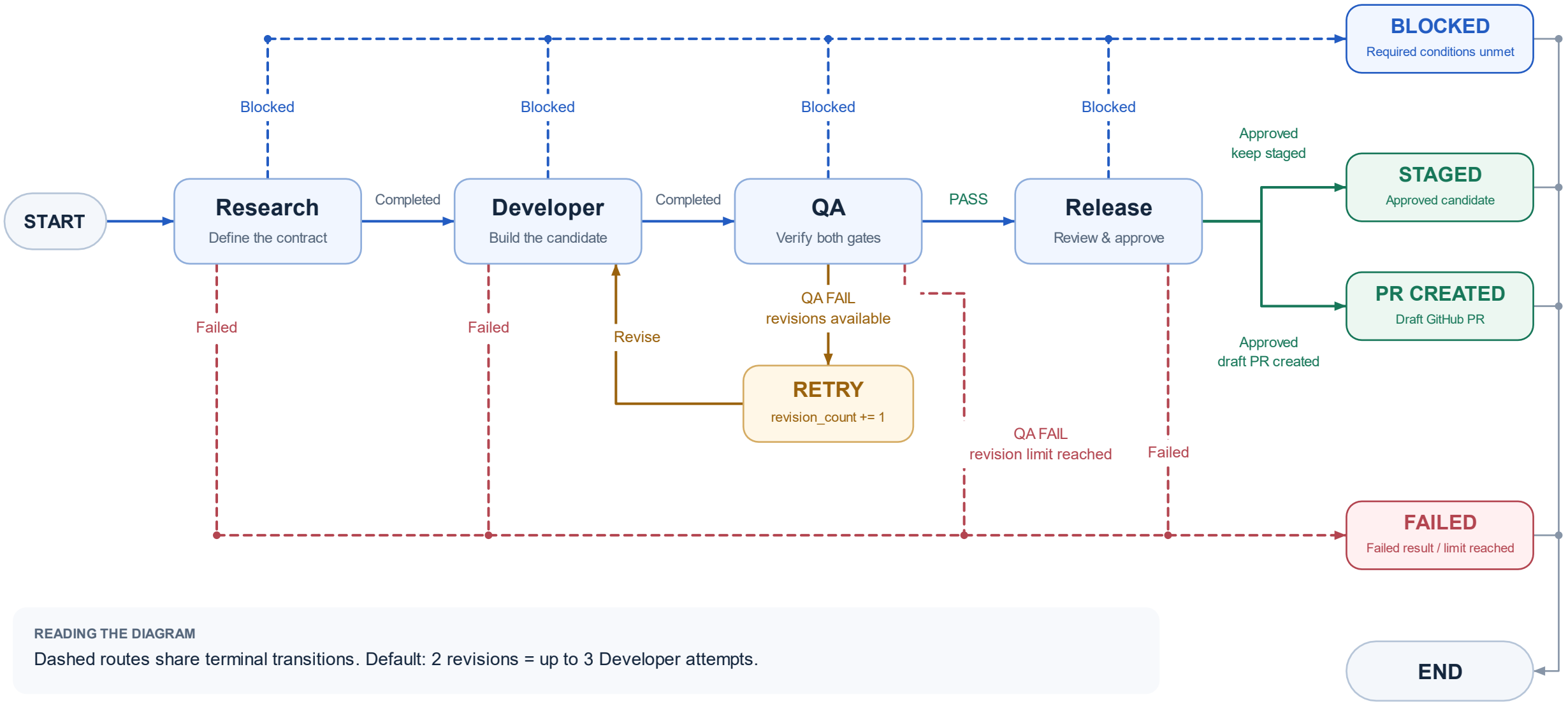
- **Input:** a natural language task, optionally accompanied by an existing repository and documentation URLs.
- **Approach:** one role-specific agent per stage, connected through typed state and saved artifacts.
- **Core design:** agents interpret and generate; deterministic code validates, executes, and routes the workflow.
- **Result:** an approved release candidate or draft PR, with clear failure and blocked outcomes.



AGENTIC HARNESS / CONTROL FLOW

A bounded state machine for agentic delivery

Four primary stages · one revision loop · four terminal outcomes



READING THE DIAGRAM

Dashed routes share terminal transitions. Default: 2 revisions = up to 3 Developer attempts.

Source: src/harness/graph.py · Human approval is enforced inside Release · Research is not repeated on retry

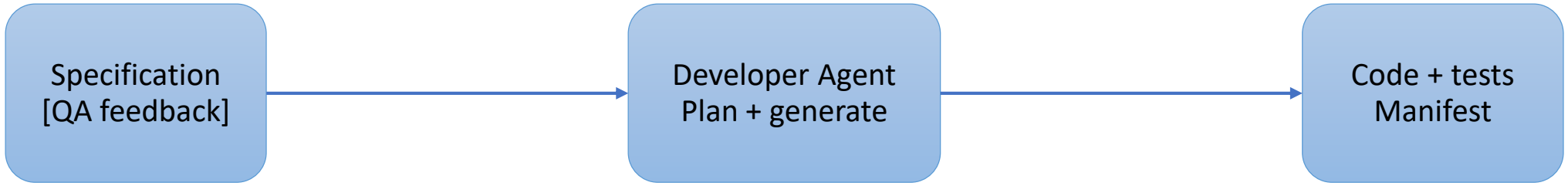
Research Harness: create the engineering contract

- **Input:** task, optional repository context, and supplied documentation URLs.
- **Research Agent:** define requirements, technical approach, implementation steps, assumptions, constraints, and acceptance criteria.
- **Harness tools:** inspect repository files, retrieve supplied documentation, validate the structured specification and its source references.
- **Output:** engineering_spec.md, engineering_spec.json, and evidence.json.
- **Gate:** continue when the specification is ready; block when essential information is missing.



Developer Harness: plan, then generate a candidate

- **Input:** specification and repository context; retries add QA feedback and the previous candidate.
- **Developer Agent, plan:** propose files, CREATE/MODIFY operations, and dependencies.
- **Developer Agent, generate:** produce complete code and tests sequentially from the plan.
- **Harness tools:** validate paths and operations, verify originals, and stage an isolated candidate.
- **Output:** code, tests, and a hashed manifest.json.



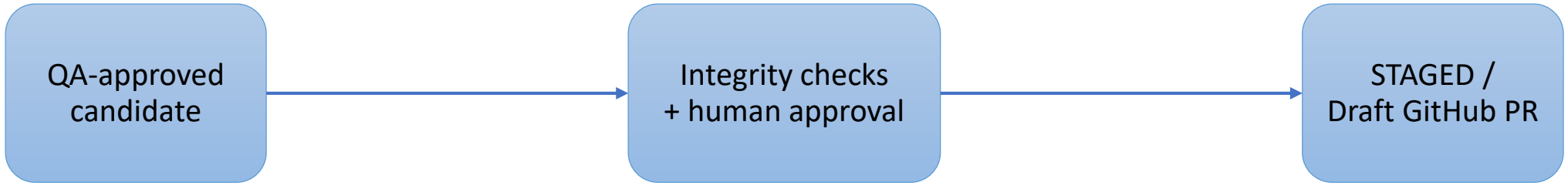
QA Harness: execution evidence and requirements review

- **Input:** specification, implementation manifest, staged files, and dependencies.
- **Execution gate:** verify hashes; run Python syntax checks and pytest in a disposable Docker workspace.
- **QA Agent:** review every requirement and acceptance criterion; cite evidence and explain repairs.
- **PASS requires both:** successful execution and all requirements assessed as satisfied.
- **Output:** test_report.json, review context, requirements findings, and structured repair feedback.



Release Harness: prepare a candidate for approval

- **Input:** the current attempt's implementation and persisted QA PASS evidence.
- **Harness gates:** revalidate QA checks, requirement coverage, file hashes, and workspace consistency; prepare an isolated candidate.
- **Release Agent:** write the change summary, testing notes, limitations, and review considerations.
- **Human decision:** retain approved output as STAGED or publish a branch and create a draft GitHub PR.
- **Output:** candidate, release report, recorded decision/result, and an optional PR URL.



Agents reason. The harness validates and executes.

Task + optional repository + supplied documentation → inspectable artifacts at every handoff

	01 Research	02 Developer	03 QA	04 Release
AGENT Model reasoning	Define requirements Choose technical approach Set acceptance criteria	Plan files & dependencies Generate code and tests Use feedback to revise	Review requirement evidence Assess coverage and gaps Explain needed repairs	Summarize changes Describe testing & limits Prepare review notes
HARNESS Deterministic tools	Inspect bounded repo context Retrieve supplied doc URLs Validate spec & source IDs	Validate paths & operations Stage complete candidate Hash files & originals	Prepare isolated workspace Run syntax + pytest in Docker Validate coverage & verdict	Recheck QA & file integrity Prepare candidate + approval Stage or publish draft PR
ARTIFACTS Saved per run / attempt	engineering_spec.md / .json evidence.json	attempt-N/staging/ manifest.json	test_report.json implementation_context.json	candidate/ + release report decision / result + optional PR

QA FAIL → Retry → Developer

The next attempt receives structured QA feedback + the verified previous candidate. Research/specification stay fixed.

One role-specific agent per stage · Shared provider abstraction · Typed state + validated artifacts + live progress

Known limitations: next improvements

Current limitation	Next improvement
Python/pytest execution runs offline; live platform behavior is unverified.	Independent acceptance tests and controlled platform integration tests.
Repository and documentation context are bounded; documentation URLs are supplied.	Task-directed retrieval, richer document parsing, and larger-repository support.
Valid schemas do not ensure correct reasoning or complete code.	Broader evaluations, bounded response repair, and stronger behavioral checks.
Stopped runs cannot resume the synchronous approval step.	Durable checkpoints and resumable human approval.

Conclusion: an end-to-end four-stage harness

- Research specification leads to generated implementation, which is verified through QA evidence and final output is human-approved release.
- Bounded repair loop and explicit FAILED / BLOCKED outcomes.
- Inspectable artifacts, typed contracts, provider adapters, and live progress.
- Recorded demo: one successful end-to-end run.

The central idea is to give agents clear responsibilities and make every handoff explicit, validated, and reviewable.



Thank you

Any Questions?